[CHAPTER] 7

SIMPLE GRAPHICS AND Image Processing

EIU
EASTERN INTERNATIONAL UNIVERSITY

Image Processing

Sampling and Digitizing Images

A visual scene projects an infinite set of color and intensity values onto a two-dimensional sensing medium, such as a human being's retina or a scanner's surface. If you sample enough of these values, the digital information can represent an image that is more or less indistinguishable to the human eye from the original scene.

Sampling devices measure discrete color values at distinct points on a two-dimensional **grid**. These values are pixels, which were introduced earlier in this chapter. In theory, the more pixels that are sampled, the more continuous and realistic the resulting image will appear. In practice, however, the human eye cannot discern objects that are closer together than 0.1 mm, so a sampling of 10 pixels per linear millimeter (250 pixels per inch and 62,500 pixels per square inch) would be plenty accurate. Thus, a 3-inch by 5-inch image would need

$$3 * 5 * 62{,}500 \text{ pixels/inch}^2 = 937{,}500 \text{ pixels}$$

which is approximately one megapixel. For most purposes, however, you can settle for a much lower sampling size and, thus, fewer pixels per square inch.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Image File Formats

Once an image has been sampled, it can be stored in one of many file formats. A **raw image file** saves all of the sampled information. This has a cost and a benefit: The benefit is that the display of a raw image will be the most true to life, but the cost is that the file size of the image can be quite large. Two of the most popular image file formats are JPEG (Joint Photographic Experts Group) and GIF (Graphic Interchange Format).

Image-Manipulation Operations

Image-manipulation programs either transform the information in the pixels or alter the arrangement of the pixels in the image.

- Rotate an image
- Convert an image from color to grayscale
- Apply color filtering to an image
- Highlight a particular area in an image
- Blur all or part of an image
- Sharpen all or part of an image
- Control the brightness of an image
- Perform edge detection on an image
- Enlarge or reduce an image's size
- Apply color inversion to an image
- Morph an image into another image

EIU
EASTERN INTERNATIONAL UNIVERSITY

The Properties of Images

When an image is loaded into a program such as a Web browser, the software maps the bits from the image file into a rectangular area of colored pixels for display. The coordinates of the pixels in this two-dimensional grid range from (0, 0) at the upper-left corner of an image to (*width* – 1, *height* – 1) at the lower-right corner, where *width* and *height* are the image's dimensions in pixels. Thus, the **screen coordinate system** for the display of an image is somewhat different from the standard Cartesian coordinate system that we used with Turtle graphics, where the origin (0, 0) is at the center of the rectangular grid. The RGB color system introduced earlier in this chapter is a common way of representing the colors in images. For our purposes, then, an image consists of a width, a height, and a set of color values accessible by means of (*x*, *y*) coordinates. A color value consists of the tuple (*r*, *g*, *b*), where the variables refer to the integer values of its red, green, and blue components, respectively.

The `images` Module

The **`images`** module is a non-standard, open-source Python tool.
but placing the file **`images.py`** and some sample image files in
your current working directory will get you started.

The **`images`** module includes a class named **`Image`**. The **`Image`** class represents an image as a two-dimensional grid of RGB values. The methods for the **`Image`** class are listed in Table 7-4. In this table, the variable **`i`** refers to an instance of the **`Image`** class.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Table 7-4 The **Image** methods

Image Method	What It Does
`i = Image(filename)`	Loads and returns an image from a file with the given filename. Raises an error if the filename is not found or the file is not a GIF file.
`i = Image(width, height)`	Creates and returns a blank image with the given dimensions. The color of each pixel is transparent, and the filename is the empty string.
`i.getWidth()`	Returns the width of `i` in pixels.
`i.getHeight()`	Returns the height of `i` in pixels.
`i.getPixel(x, y)`	Returns a tuple of integers representing the RGB values of the pixel at position (x, y).
`i.setPixel(x, y, (r, g, b))`	Replaces the RGB value at the position (x, y) with the RGB value given by the tuple `(r, g, b)`.
`i.draw()`	Displays `i` in a window. The user must close the window to return control to the method's caller.
`i.clone()`	Returns a copy of `i`.
`i.save()`	Saves `i` under its current filename. If `i` does not yet have a filename, `save` does nothing.
`i.save(filename)`	Saves `i` under `filename`. Automatically adds a `.gif` extension if `filename` does not contain it.

Before we discuss some standard image-processing algorithms, let's try out the resources of the **images** module. This version of the **images** module accepts only image files in GIF format. For the purposes of this exercise, we also assume that a GIF image of my cat, Smokey, has been saved in a file named **smokey.gif** in the current working directory. The following session with the interpreter does three things:

1. Imports the **Image** class from the **images** module
2. Instantiates this class using the file named **smokey.gif**
3. Draws the image

The resulting image display window is shown in Figure 7-9.

```
>>> from images import Image
>>> image = Image("smokey.gif")
>>> image.draw()
```



Figure 7-9 An image display window

```
>>> print(image)
Filename: smokey.gif
Width: 198
Height: 149
```

```
>>> image.getWidth()
198
>>> image.getHeight()
149
>>> image.getPixel(0, 0)
(194, 221, 114)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> image = Image(150, 150)
>>> image.draw()
>>> blue = (0, 0, 255)
>>> y = image.getHeight() // 2
>>> for x in range(image.getWidth()):
        image.setPixel(x, y - 1, blue)
        image.setPixel(x, y, blue)
        image.setPixel(x, y + 1, blue)
>>> image.draw()
```



Figure 7-10 An image before and after replacing the pixels

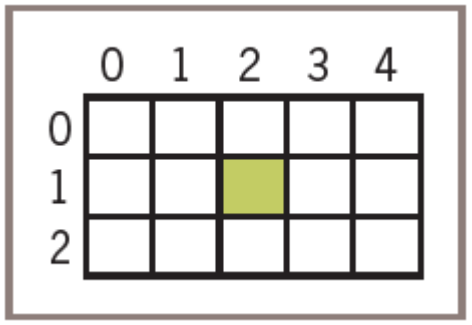
Finally, you can save an image under its current filename or a different filename. Use the **save** operation to write an image back to an existing file using the current filename. The **save** operation can also receive a string parameter for a new filename. The image is written to a file with that name, which then becomes the current filename. The following code saves the new image using the filename **horizontal.gif**:

```
>>> image.save("horizontal.gif")
```

If you omit the **.gif** extension in the filename, the method adds it automatically.

EIU EASTERN INTERNATIONAL UNIVERSITY

A Loop Pattern for Traversing a Grid



By contrast, many image-processing algorithms use a **nested loop structure** to traverse a two-dimensional grid of pixels. Figure 7-11 shows such a grid. Its height is 3 rows, numbered 0 through 2. Its width is 5 columns, numbered 0 through 4. Each data value in the grid is accessed with a pair of coordinates using the form `(<column>, <row>)`. Thus, the datum in the middle of the grid, which is shaded, is at position (2, 1). The datum in the upper-left corner is at the origin of the grid, (0, 0).

Figure 7-11 A grid with 3 rows and 5 columns

A nested loop structure to traverse a grid consists of two loops, an outer one and an inner one.

```
>>> width = 2
>>> height = 3
>>> for y in range(height):
        for x in range(width):
            print((x, y), end = " ")
        print()
(0, 0) (1, 0)
(0, 1) (1, 1)
(0, 2) (1, 2)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Converting an Image to Black and White

```
def blackAndWhite(image):
    """Converts the argument image to black and white."""
    blackPixel = (0, 0, 0)
    whitePixel = (255, 255, 255)
    for y in range(image.getHeight()):
        for x in range(image.getWidth()):
            (r, g, b) = image.getPixel(x, y)
            average = (r + g + b) // 3
            if average < 128:
                image.setPixel(x, y, blackPixel)
            else:
                image.setPixel(x, y, whitePixel)
```

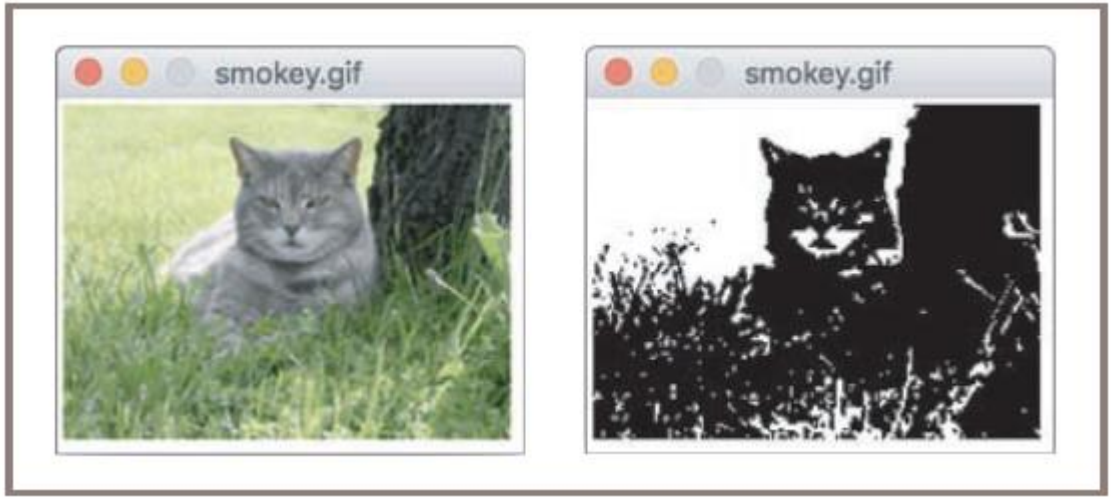


Figure 7-12 Converting a color image to black and white

The function can be tested in a short script, as follows:

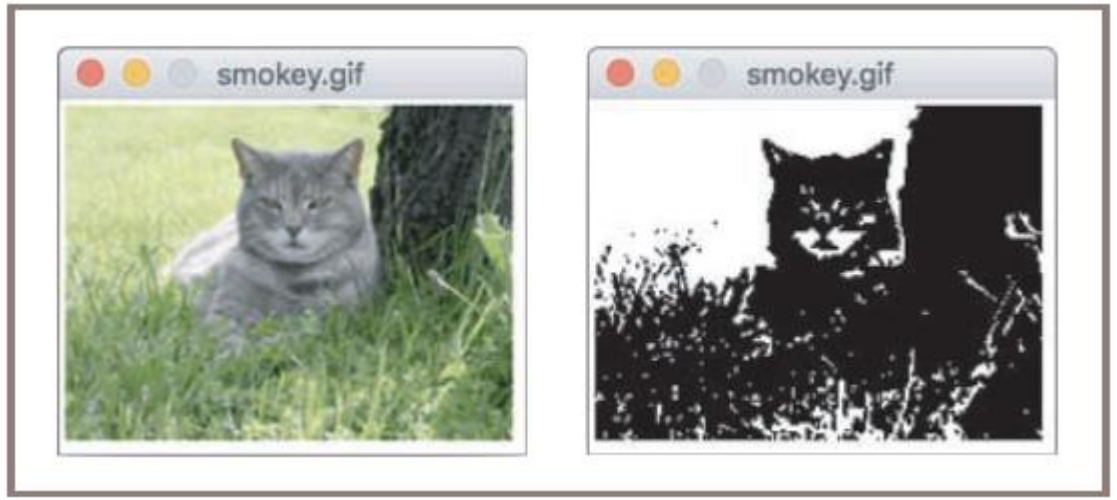
```
from images import Image

# Code for blackAndWhite's function definition goes here

def main(filename = "smokey.gif"):
    image = Image(filename)
    print("Close the image window to continue.")
    image.draw()
    blackAndWhite(image)
    print("Close the image window to quit.")
    image.draw()

if __name__ == "__main__":
    main()
```

Figure 7-12 Converting a color image to black and white



Note that the **main** function includes an optional argument for the image filename. Its default should be the name of an image in the current working directory.

Converting an Image to Grayscale

Black-and-white photographs are not really just black and white; they also contain various shades of gray known as **grayscale**. Grayscale can be an economical color scheme, wherein the only color values might be 8, 16, or 256 shades of gray (including black and white at the extremes).

```
def grayscale(image):
    """Converts the argument image to grayscale."""
    for y in range(image.getHeight()):
        for x in range(image.getWidth()):
        (r, g, b) = image.getPixel(x, y)
        r = int(r * 0.299)
        g = int(g * 0.587)
        b = int(b * 0.114)
        lum = r + g + b
        image.setPixel(x, y, (lum, lum, lum))
```

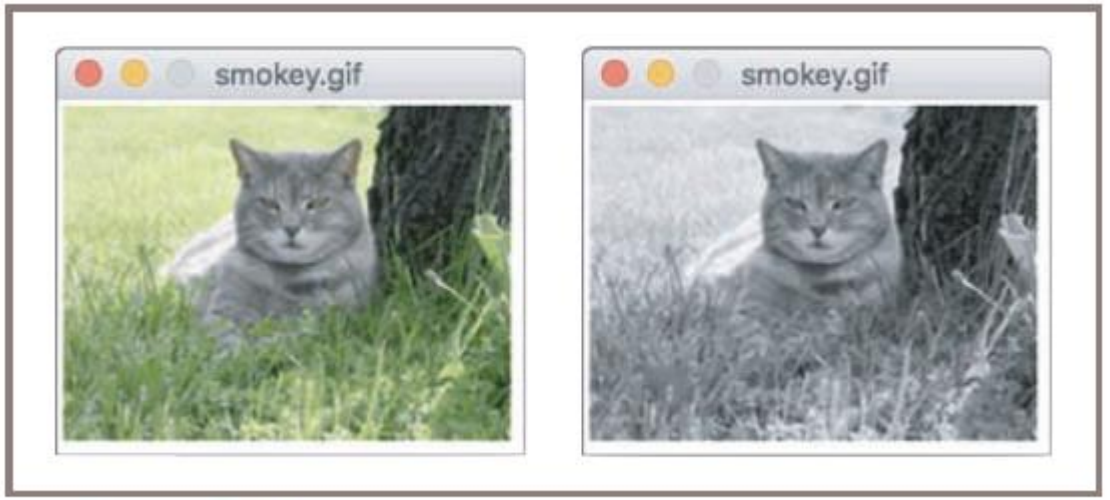


Figure 7-13 Converting a color image to grayscale

EIU
EASTERN INTERNATIONAL UNIVERSITY

Copying an Image

The method **clone** builds and returns a new image with the same attributes as the original one, but with an empty string as the filename.

```
>>> from images import Image
>>> image = Image("smokey.gif")
>>> image.draw()
>>> newImage = image.clone()    # Create a copy of image
>>> newImage.draw()
>>> grayscale(newImage)         # Change in second window only
>>> newImage.draw()
>>> image.draw()                # Verify no change to original
```

Blurring an Image

Occasionally, an image appears to contain rough, jagged edges. This condition, known as **pixilation**, can be mitigated by blurring the image's problem areas. **Blurring** makes these areas appear softer, but at the cost of losing some definition. We now develop a simple algorithm to blur an entire image. This algorithm resets each pixel's color to the average of the colors of the four pixels that surround it. The function **blur** expects an image as an argument and returns a copy of that image with blurring. The function **blur** begins its traversal of the grid with position (1, 1) and ends with position (*width* − 2, *height* − 2). This means that the algorithm does not transform the pixels on the image's outer edges.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
def blur(image):
    """Builds and returns a new image which is a
    blurred copy of the argument image."""

    def tripleSum(triple1, triple2):
#1
        (r1, g1, b1) = triple1
        (r2, g2, b2) = triple2
        return (r1 + r2, g1 + g2, b1 + b2)

    new = image.clone()
    for y in range(1, image.getHeight() - 1):
        for x in range(1, image.getWidth() - 1):
            oldP = image.getPixel(x, y)
            left = image.getPixel(x - 1, y)     # To left
            right = image.getPixel(x + 1, y)    # To right
            top = image.getPixel(x, y - 1)      # Above
            bottom = image.getPixel(x, y + 1)   # Below
            sums = reduce(tripleSum,
                          [oldP, left, right, top, bottom])
#2
            averages = tuple(map(lambda x: x // 5, sums))
#3
            new.setPixel(x, y, averages)
    return new
```

- At #1, the nested auxiliary function **tripleSum** is defined. This function expects two tuples of integers as arguments and returns a single tuple containing the sums of the values at each position.
- At #2, five tuples of RGB values are wrapped in a list and passed with the **tripleSum** function to the **reduce** function. This function repeatedly applies **tripleSum** to compute the sums of the tuples, until a single tuple containing the sums is returned.
- At #3, a **lambda** function is mapped onto the tuple of sums, and the result is converted to a tuple. The **lambda** function divides each sum by 5. Thus, you are left with a tuple of the average RGB values.

Edge Detection

When artists paint pictures, they often sketch an outline of the subject in pencil or charcoal. They then fill in and color over the outline to complete the painting. **Edge detection** performs the inverse function on a color image: It removes the full colors to uncover the outlines of the objects represented in the image.

A simple edge-detection algorithm examines the neighbors below and to the left of each pixel in an image. If the luminance of the pixel differs from that of either of these two neighbors by a significant amount, you have detected an edge, and you set that pixel's color to black. Otherwise, you set the pixel's color to white.

The function **`detectEdges`** expects an image and an integer as parameters. The function returns a new black-and-white image that explicitly shows the edges in the original image. The integer parameter allows the user to experiment with various differences in luminance.



Figure 7-14

Edge detection: the original image, a luminance threshold of 10, and a luminance threshold of 20

```
def detectEdges(image, amount):
    """Builds and returns a new image in which the edges of
    the argument image are highlighted and the colors are
    reduced to black and white."""

    def average(triple):
        (r, g, b) = triple
        return (r + g + b) // 3

    blackPixel = (0, 0, 0)
    whitePixel = (255, 255, 255)
    new = image.clone()
    for y in range(image.getHeight() - 1):
        for x in range(1, image.getWidth()):
            oldPixel = image.getPixel(x, y)
            leftPixel = image.getPixel(x - 1, y)
            bottomPixel = image.getPixel(x, y + 1)
            oldLum = average(oldPixel)
            leftLum = average(leftPixel)
            bottomLum = average(bottomPixel)
            if abs(oldLum - leftLum) > amount or \
               abs(oldLum - bottomLum) > amount:
                new.setPixel(x, y, blackPixel)
            else:
                new.setPixel(x, y, whitePixel)
    return new
```

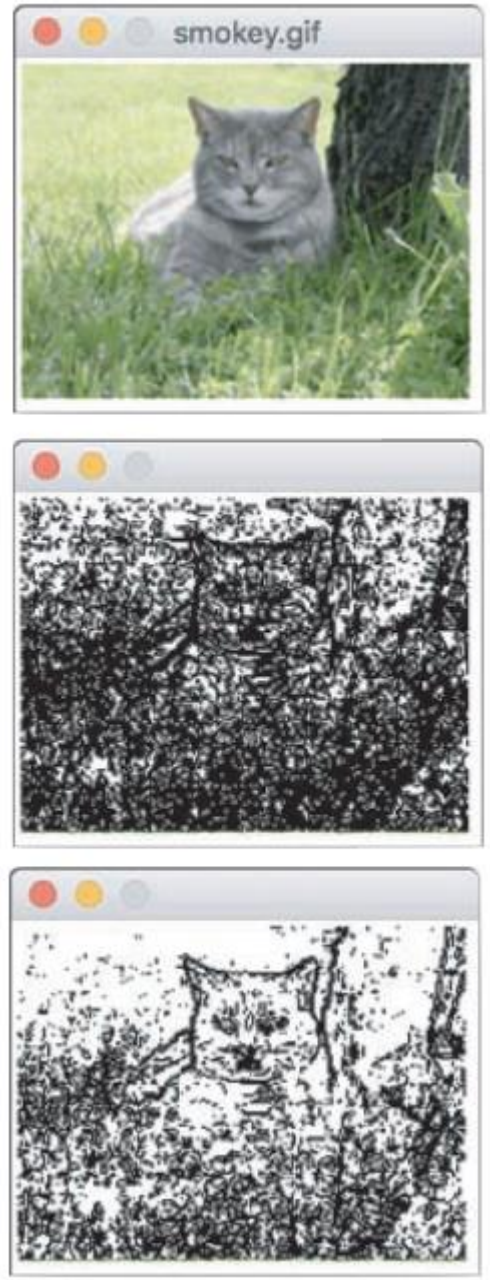


Figure 7-14

Edge detection: the original image, a luminance threshold of 10, and a luminance threshold of 20

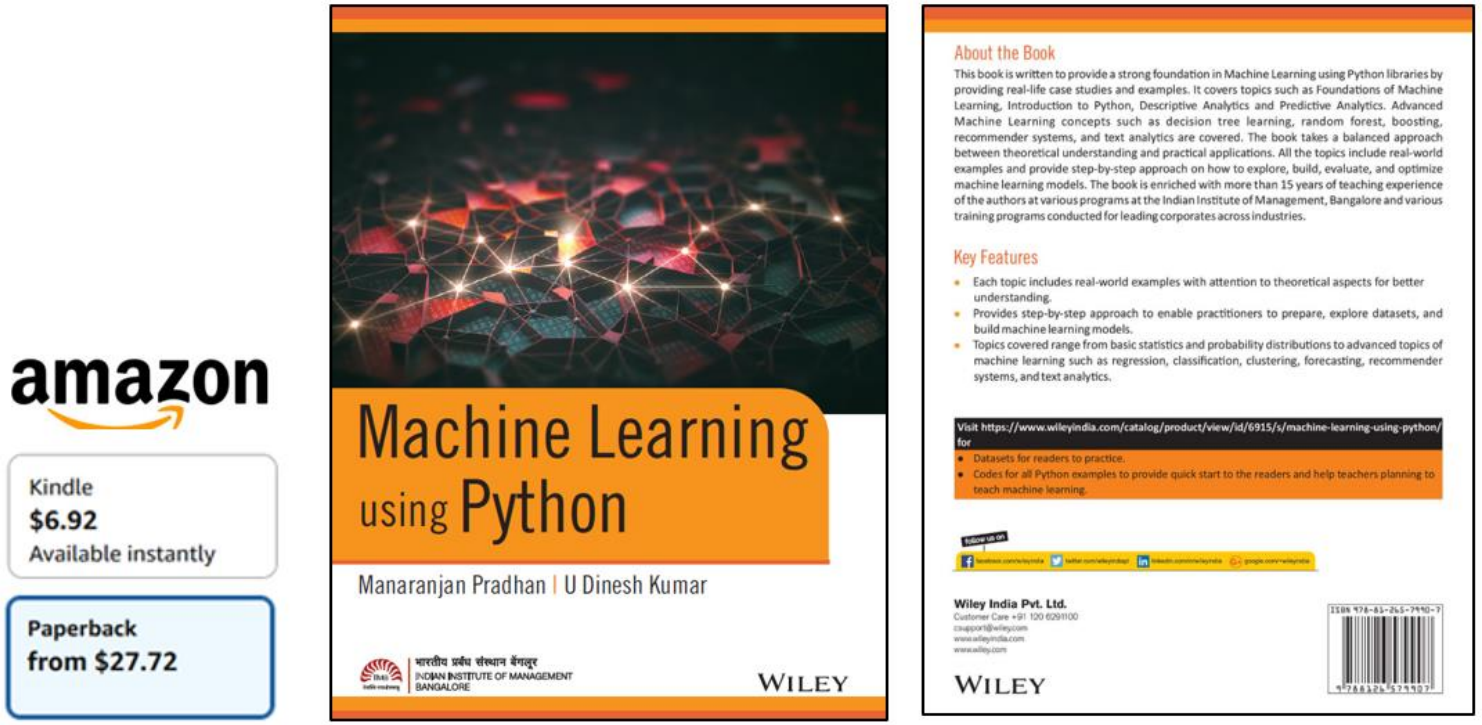
Reducing the Image Size

A size reduction usually preserves an image's **aspect ratio** (that is, the ratio of its width to its height). A simple way to shrink an image is to create a new image whose width and height are a constant fraction of the original image's width and height. The algorithm then copies the color values of just some of the original image's pixels to the new image. For example, to reduce the size of an image by a factor of 2, you could copy the color values from every other row and every other column of the original image to the new image.

The Python function **shrink** exploits this strategy. The function expects the original image and a positive integer shrinkage factor as parameters.

```
def shrink(image, factor):
    """Builds and returns a new image which is a smaller
    copy of the argument image, by the factor argument."""
    width = image.getWidth()
    height = image.getHeight()
    new = Image(width // factor, height // factor)
    oldY = 0
    newY = 0
    while oldY < height - factor:
        oldX = 0
        newX = 0
        while oldX < width - factor:
            oldP = image.getPixel(oldX, oldY)
            new.setPixel(newX, newY, oldP)
            oldX += factor
            newX += 1
        oldY += factor
        newY += 1
    return new
```

EXPLORATION OF DATA USING VISUALIZATION



EIU
EASTERN INTERNATIONAL UNIVERSITY

2.3.1 | Drawing Plots

Matplotlib is a Python 2D plotting library and most widely used library for data visualization. It provides extensive set of plotting APIs to create various plots such as scattered, bar, box, and distribution plots with custom styling and annotation. Detailed documentation for *matplotlib* can be found at *https://matplotlib.org/*. Seaborn is also a Python data visualization library based on matplotlib. It provides a high-level interface for drawing innovative and informative statistical charts (source: *https://seaborn.pydata.org/*).

Matplotlib is a library for creating 2D plots of arrays in Python. Matplotlib is written in Python and makes use of NumPy arrays. It is well integrated with pandas to read columns and create plots. *Seaborn*, which is built on top of matplotlib, is a library for making elegant charts in Python and is well integrated with pandas DataFrame.

To create graphs and plots, we need to import *matplotlib.pyplot* and *seaborn* modules. To display the plots on the Jupyter Notebook, we need to provide a directive *%matplotlib inline*. Only if the directive is provided, the plots will be displayed on the notebook.

```
import matplotlib.pyplot as plt
import seaborn as sn
%matplotlib inline
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

2.3.2 | Bar Chart

Bar chart is a frequency chart for qualitative variable (or categorical variable). Bar chart can be used to assess the most-occurring and least-occurring categories within a dataset.

To draw a bar chart, call *barplot()* of seaborn library. The DataFrame should be passed in the parameter *data*. To display the average sold price by each age category, pass *SOLD PRICE* as *y* parameter and *AGE* as *x* parameter. Figure 2.3 shows a bar plot created to show the average SOLD PRICE for each age category.

```
sn.barplot(x = 'AGE', y = 'SOLD PRICE', data = soldprice_by_age);
```

In Figure 2.3, it can be noted that the average sold price is higher for age category 1.We can also create bar charts, which are grouped by a third variable.

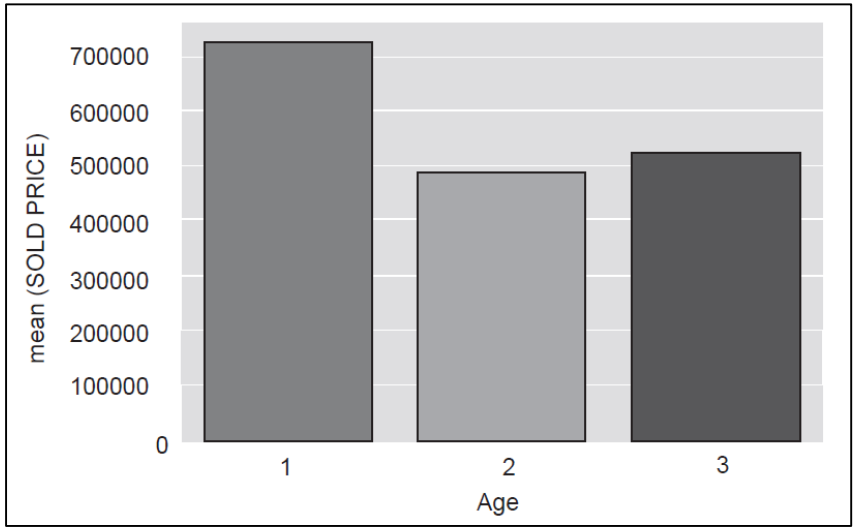
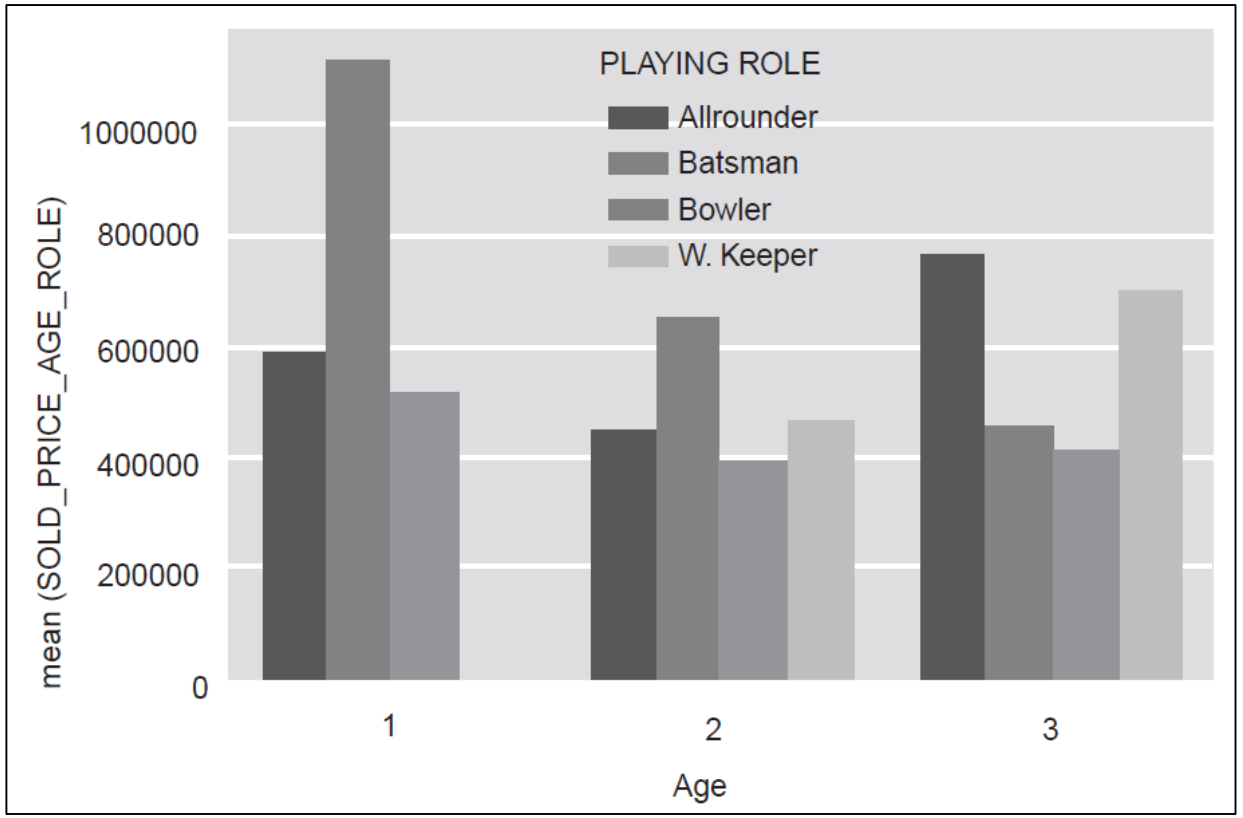


FIGURE 2.3 Bar plot for average sold price versus age.

```
sn.barplot(x = 'AGE', y = 'SOLD_PRICE_AGE_ROLE', hue = 'PLAYING
ROLE', data = soldprice_comparison);
```



In the following example (Figure 2.4), average sold price is shown for each age category but grouped by a third variable, that is, playing roles. The parameter *hue* takes the third variable as parameter. In this case, we pass *PLAYING ROLE* as *hue* parameter.

In Figure 2.4, it can be noted that in the age categories 1 and 2, batsmen are paid maximum, whereas allrounders are paid maximum in the age category 3. This could be because allrounders establish their credentials as good allrounders over a period.

FIGURE 2.4 Bar plot for average sold price versus age by player roles. Here the first bar represents Allrounder, the second Batsman, the third Bowler, the fourth W. Keeper.

2.3.3 | Histogram

A **histogram** is a plot that shows the frequency distribution of a set of continuous variable. Histogram gives an insight into the underlying distribution (e.g., normal distribution) of the variable, outliers, skewness, etc. To draw a histogram, invoke *hist()* method of matplotlib library. The following is an example of how to draw a histogram for *SOLD PRICE* and understand its distribution (Figure 2.5).

```
plt.hist( ipl_auction_df['SOLD PRICE'] );
```

The histogram shows that *SOLD PRICE* is right skewed. Most players are auctioned at low price range of 250000 and 500000, whereas few players are paid very highly, more than 1 million dollars.

By default, it creates 10 bins in the histogram. To create more bins, the bins parameter can be set in the *hist()* method as follows:

```
plt.hist( ipl_auction_df['SOLD PRICE'], bins = 20 );
```

Histogram for SOLD PRICE with 20 bins is shown in Figure 2.6.

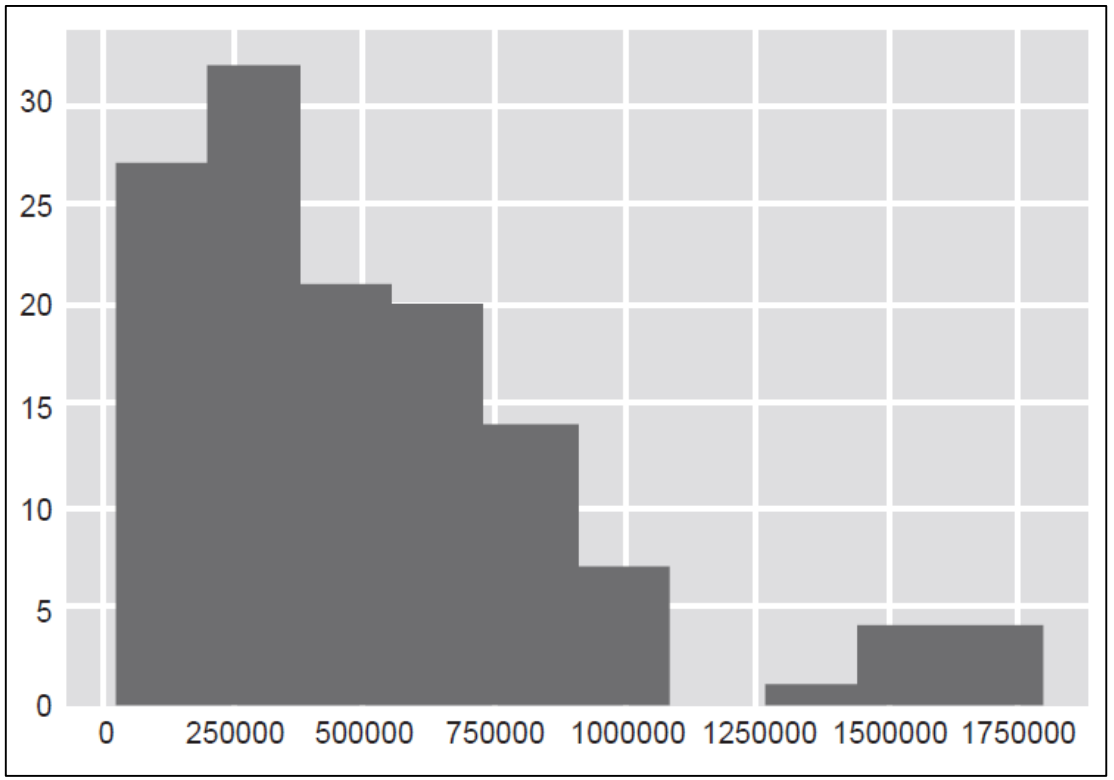


FIGURE 2.5 Histogram for SOLD PRICE.

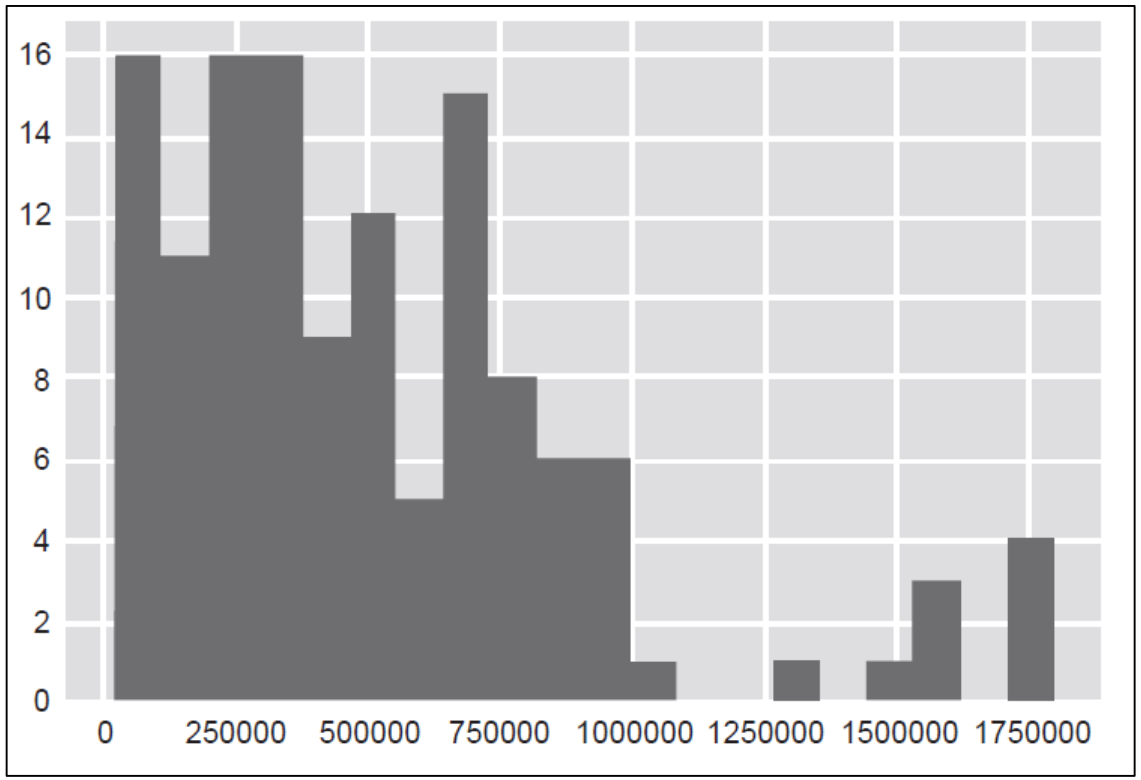


FIGURE 2.6 Histogram for SOLD PRICE with 20 bins.

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

2.3.4 | Distribution or Density Plot

A **distribution or density** plot depicts the distribution of data over a continuous interval. Density plot is like smoothed histogram and visualizes distribution of data over a continuous interval. So, a density plot also gives insight into what might be the distribution of the population.

```
sn.distplot(ipl_auction_df['SOLD PRICE']);
```

To draw the distribution plot, we can use *distplot()* of seaborn library. Density plot for the outcome variable "SOLD PRICE" is shown in Figure 2.7.

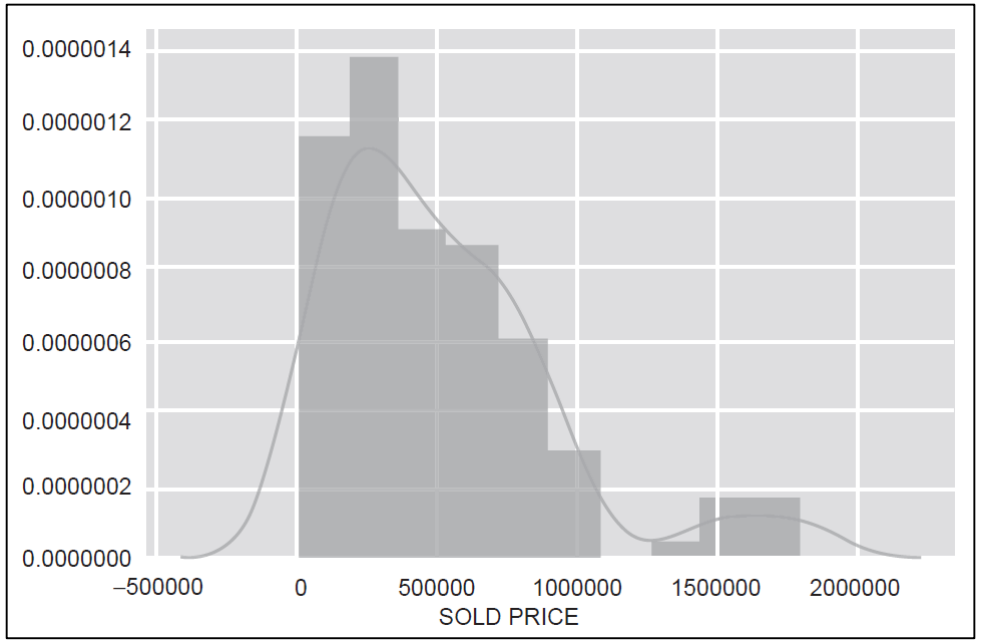


FIGURE 2.7 Distribution plot for SOLD PRICE.

EIU EASTERN INTERNATIONAL UNIVERSITY

2.3.5 | Box Plot

Box plot (aka Box and Whisker plot) is a graphical representation of numerical data that can be used to understand the variability of the data and the existence of outliers. Box plot is designed by identifying the following descriptive statistics:

1. Lower quartile (1st quartile), median and upper quartile (3rd quartile).
2. Lowest and highest values.
3. Inter-quartile range (IQR).

Box plot is constructed using IQR, minimum and maximum values. IQR is the distance (difference) between the 3rd quartile and 1st quartile. The length of the box is equivalent to IQR. It is possible that the data may contain values beyond Q1 – 1.5IQR and Q3 + 1.5IQR. The whisker of the box plot extends till Q1 – 1.5IQR and Q3 + 1.5IQR; observations beyond these two limits are potential outliers.

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

To draw the box plot, call *boxplot()* of seaborn library. The box plot for SOLD PRICE is shown in Figure 2.8 and indicates that there are few outliers in the data.

```
box = sn.boxplot(ipl_auction_df['SOLD PRICE']);
```

To obtain min, max, 25 percentile (1st quartile) and 75 percentile (3rd quantile) values in the boxplot, *boxplot()* method of matplotlib can be used. Let us assign the return value of boxplot() to a variable *box*.

```
box = plt.boxplot(ipl_auction_df['SOLD PRICE']);
```

The above line of code creates a box plot as shown in Figure 2.9 and returns details of box plot in variable box. The *caps* key in box variable returns the min and max values of the distribution. Here the minimum auction price offered is 20,000 and maximum price is 13,50,000.

```
[item.get_ydata()[0] for item in box['caps']]
```

```
[20000.0, 1350000.0]
```

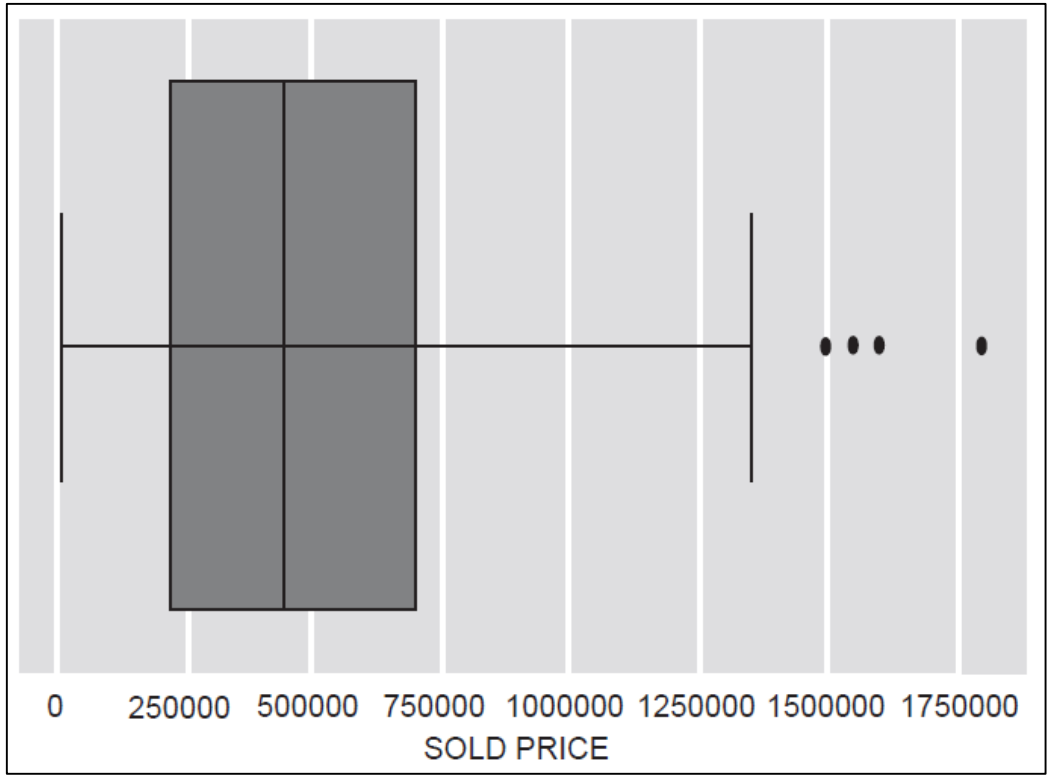


FIGURE 2.8 Box plot for SOLD PRICE.

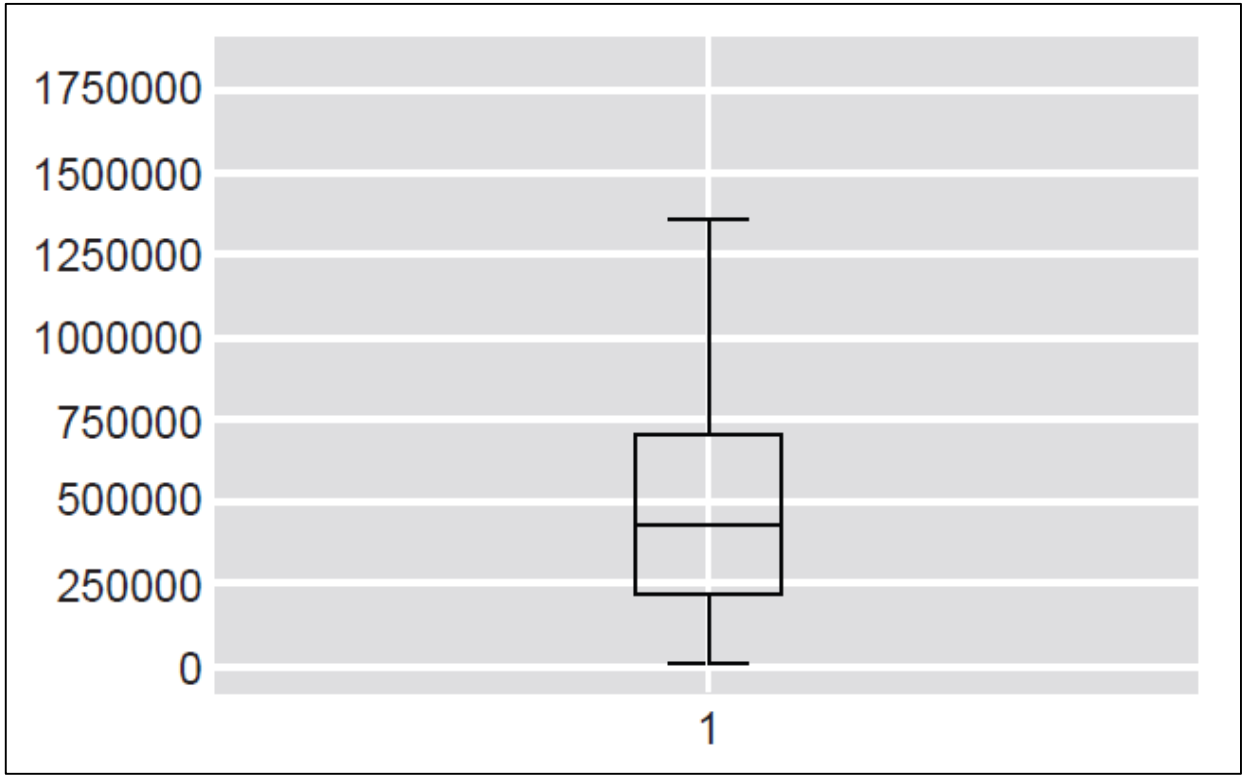


FIGURE 2.9 Box plot for SOLD PRICE with median line.

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

The *whiskers* key in box variable returns the values of the distribution at 25 and 75 quantiles. Here the min value is 225,000 and max is 700,000.

```
[item.get_ydata()[0] for item in box['whiskers']]
```

```
[225000.0, 700000.0]
```

So, inter-quartile range (IQR) is 700,000 – 225,000 = 475,000.

The *medians* key in box variable returns the median value of the distribution. Here the median value is 437,500.

```
[item.get_ydata()[0] for item in box['medians']]
```

```
[437500.0]
```

The box plot in Figure 2.8 shows that some of the players are auctioned at *SOLD PRICE*, which seem to be outliers. Let us find out the player names.

```
ipl_auction_df[ipl_auction_df['SOLD PRICE'] > 1350000.0][['PLAYER
NAME', 'PLAYING ROLE', 'SOLD PRICE']]
```

	Player Name	Playing Role	Sold Price
15	Dhoni, MS	W. Keeper	1500000
23	Flintoff, A	Allrounder	1550000
50	Kohli, V	Batsman	1800000
83	Pietersen, KP	Batsman	1550000
93	Sehwag, V	Batsman	1800000
111	Tendulkar, SR	Batsman	1800000
113	Tiwary, SS	Batsman	1600000
127	Yuvraj Singh	Batsman	1800000

The SOLD PRICE amount for the above players seems to be outlier considering what is paid to other players.

2.3.6 | Comparing Distributions

The distribution for different categories can be compared by overlapping the distributions. For example, the ***SOLD PRICE* of players with and without *CAPTAINCY EXP* can be compared as below.** Distributions can be compared by drawing distribution for each category and overlapping them in one plot.

```
sn.distplot(ipl_auction_df[ipl_auction_df['CAPTAINCY EXP'] == 1]
            ['SOLD PRICE'],
            color = 'y',
            label = 'Captaincy Experience')
sn.distplot(ipl_auction_df[ipl_auction_df['CAPTAINCY EXP'] == 0]
             ['SOLD PRICE'],
             color = 'r',
             label = 'No Captaincy Experience');
plt.legend();
```

Comparison plot for SOLD PRICE with and without captaincy experience is shown in Figure 2.10. It can be observed that players with captaincy experience seem to be paid higher.

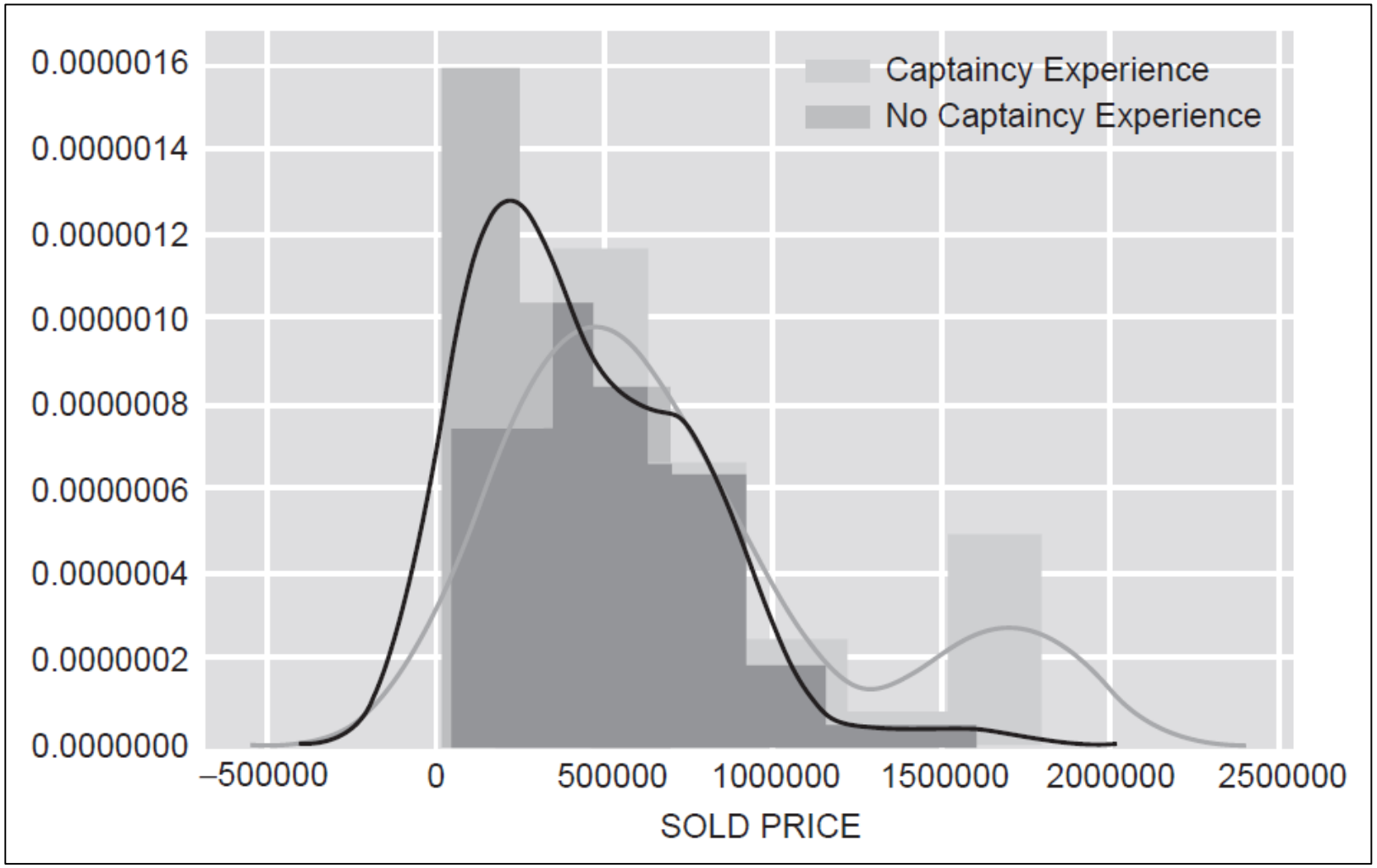


FIGURE 2.10 Distribution plots comparing SOLD PRICE with and without captaincy experience.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Similarly, the distributions can be compared using box plots. For example, if we want to visualize *SOLD PRICE* for each *PLAYING ROLE*, then pass *PLAYING ROLE* as x parameter and *SOLD PRICE* as y parameter to *boxplot()* method of seaborn.

An example of comparing the *SOLD PRICE* for players with different *PLAYING ROLE* is shown in Figure 2.11.

```
sn.boxplot(x = 'PLAYING ROLE', y = 'SOLD PRICE',
                data = ipl_auction_df);
```

Few observations from the plot in Figure 2.11 are as follows:

1. The median SOLD PRICE for allrounders and batsmen are higher than bowlers and wicket keepers.
2. Allrounders who are paid more than 1,35,0000 USD are not considered outliers. Allrounders have relatively high variance.
3. There are outliers in batsman and wicket keeper category.

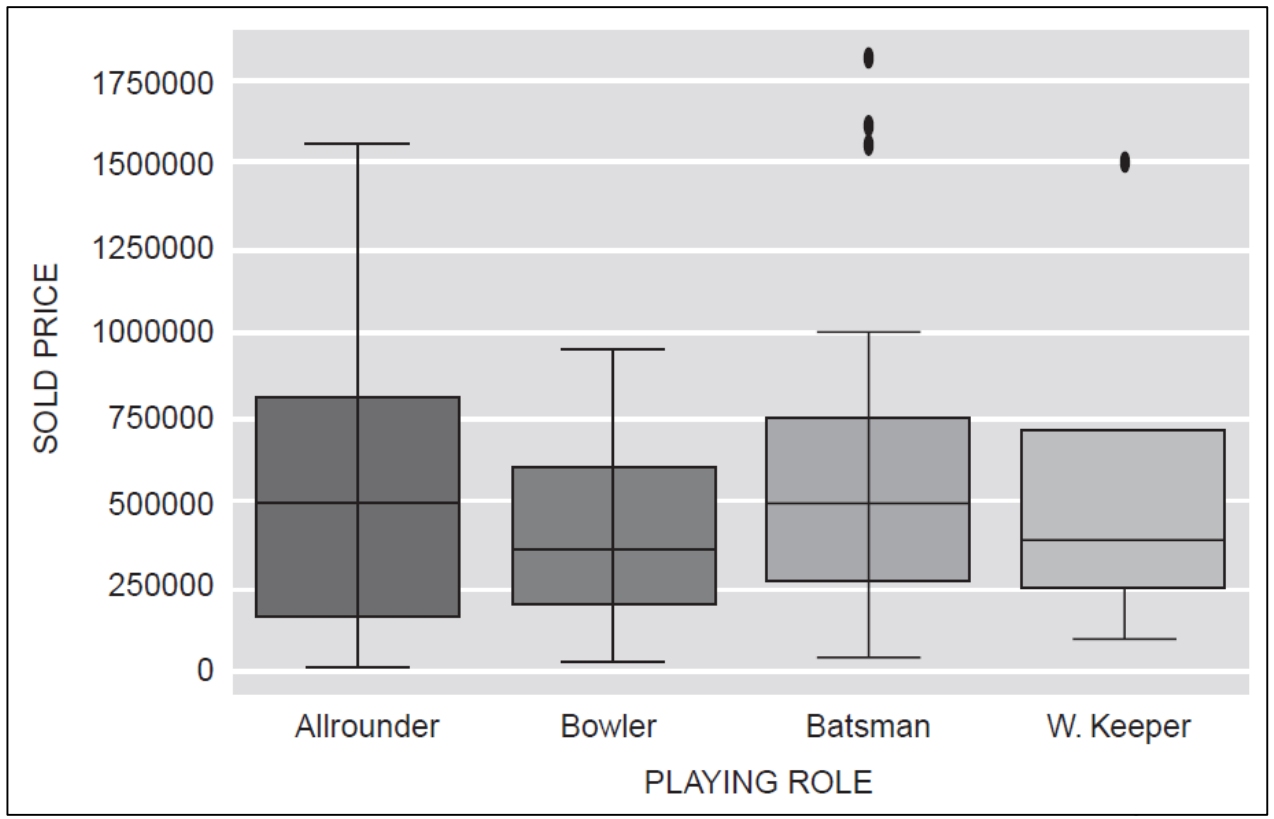


FIGURE 2.11 Box plot of SOLD PRICE for different playing roles.

2.3.7 | Scatter Plot

In a scatter plot, the values of two variables are plotted along two axes and resulting pattern can reveal correlation present between the variables, if any. The relationship could be linear or non-linear. A scatter plot is also useful for assessing the strength of the relationship and to find if there are any outliers in the data.

Scatter plots are used during regression model building to decide on the initial model, that is whether to include a variable in a regression model or not.

Since IPL is T20 cricket, it is believed that the number of sixers a player has hit in past would have influenced his SOLD PRICE. A scatter plot between *SOLD PRICE* of batsman and number of sixes the player has hit can establish this correlation. The *scatter()* method of matplotlib can be used to draw the scatter plot which takes both the variables. We will draw the scatter plot only for batsman playing role. Figure 2.12 shows the scatter plot between SIXERS and SOLD PRICE.

```
ipl_batsman_df = ipl_auction_df[ipl_auction_df['PLAYING ROLE'] ==
                 'Batsman']
```

```
plt.scatter(x = ipl_batsman_df.SIXERS,
            y = ipl_batsman_df['SOLD PRICE']);
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

To draw the direction of relationship between the variables, *regplot()* of seaborn can be used (Figure 2.13).

```
sn.regplot( x = 'SIXERS',
            y = 'SOLD PRICE',
            data = ipl_batsman_df );
```

The line in Figure 2.13 shows there is a positive correlation between number of sixes hit by a batsman and the SOLD PRICE.

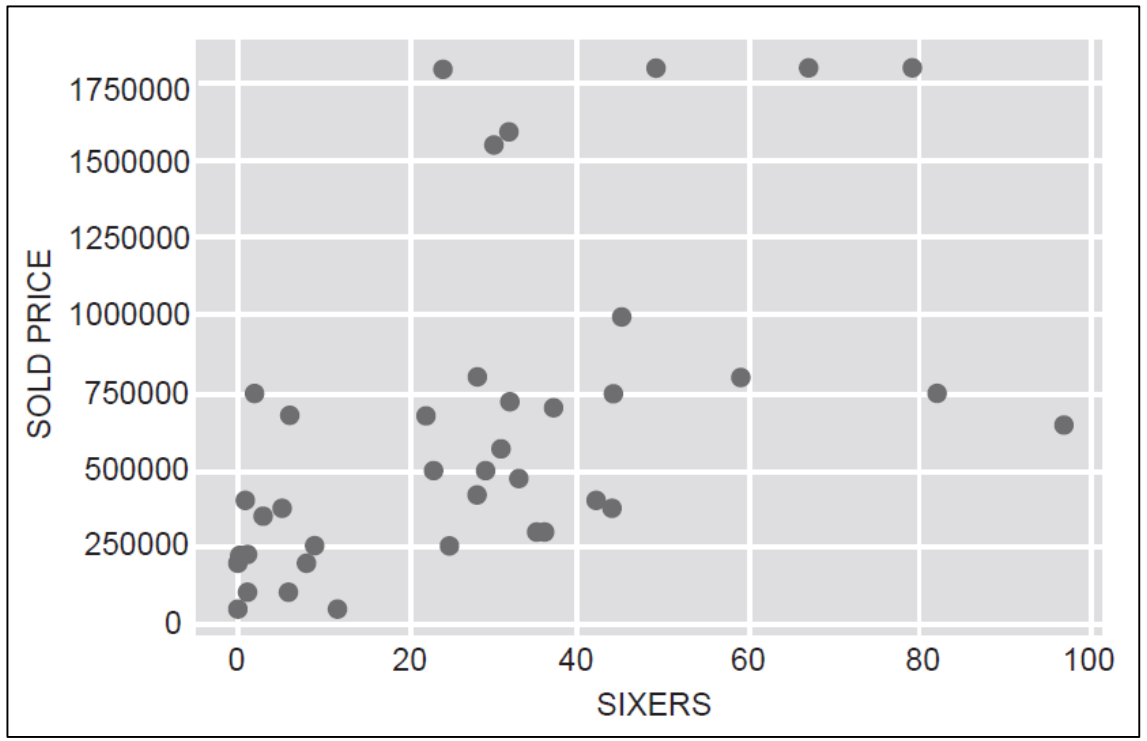


FIGURE 2.12 Scatter plot for SOLD PRICE versus SIXERS.

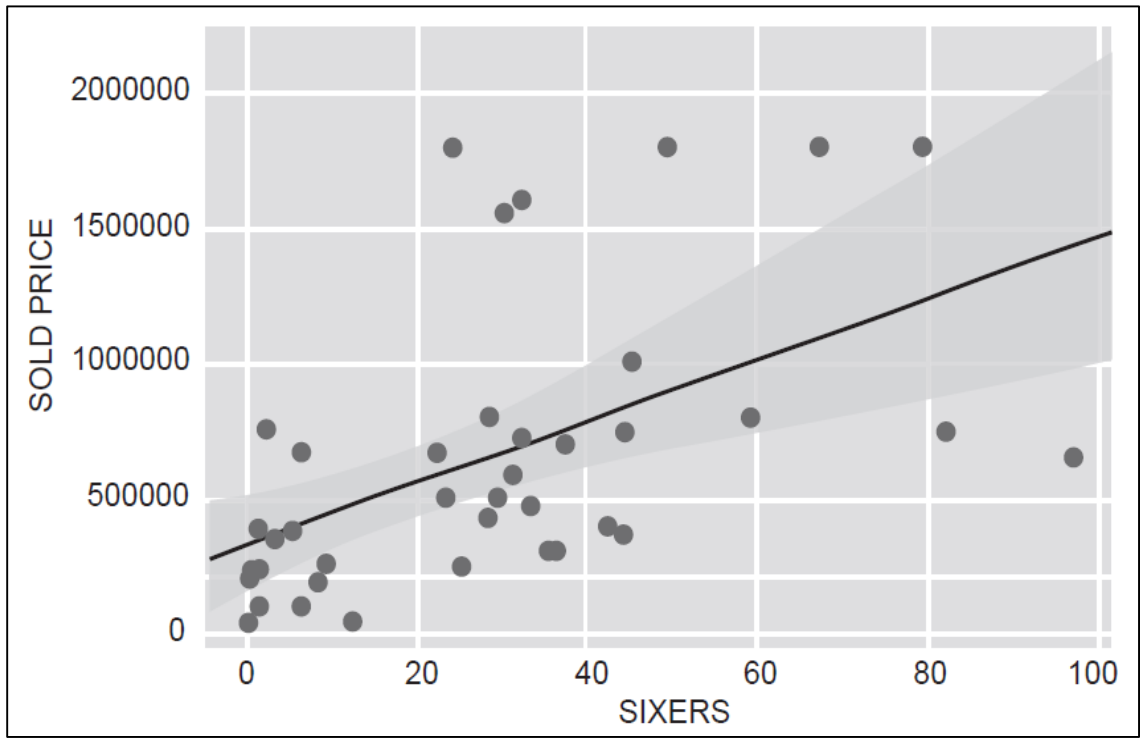


FIGURE 2.13 Scatter plot for SOLD PRICE versus SIXERS with a regression line.

2.3.8 | Pair Plot

If there are many variables, it is not convenient to draw scatter plots for each pair of variables to understand the relationships. So, a pair plot can be used to depict the relationships in a single diagram which can be plotted using *pairplot()* method.

In this example, we will explore the relationship of four variables, *SR-B, AVE, SIXERS, SOLD PRICE*, which we think may be the influential features in determining the *SOLD PRICE* of batsmen (Figure 2.14).

```
influential_features = ['SR-B', 'AVE', 'SIXERS', 'SOLD PRICE']
```

```
sn.pairplot(ipl_auction_df[influential_features], size=2)
```

The plot is drawn like a matrix and each row and column is represented by a variable. Each cell depicts the relationship between two variables, represented by that row and column variable. For example, the cell on second row and first column shows the relationship between AVE and SR-B. The diagonal of the matrix shows the distribution of the variable.

For all the correlations, *AVE* and *SIXERS* seem to be highly correlated with *SOLD PRICE* compared to *SR-B*.



FIGURE 2.14 Pair plot between variables.

2.3.9 | Correlation and Heatmap

Correlation is used for measuring the strength and direction of the linear relationship between two continuous random variables X and Y. It is a statistical measure that indicates the extent to which two variables change together. A positive correlation means the variables increase or decrease together; a negative correlation means if one variable increases, the other decreases.

1. The correlation value lies between −1.0 and 1.0. The sign indicates whether it is positive or negative correlation.
2. −1.0 indicates a perfect negative correlation, whereas +1.0 indicates perfect positive correlation.

Correlation values can be computed using *corr()* method of the DataFrame and rendered using a heatmap (Figure 2.15).

```
ipl_auction_df[influential_features].corr()
```

```
sn.heatmap(ipl_auction_df[influential_features].corr(), annot=True);
```

	SR-B	AVE	Sixers	Sold Price
SR-B	1.000000	0.583579	0.425394	0.184278
AVE	0.583579	1.000000	0.705365	0.396519
Sixers	0.425394	0.705365	1.000000	0.450609
Sold Price	0.184278	0.396519	0.450609	1.000000

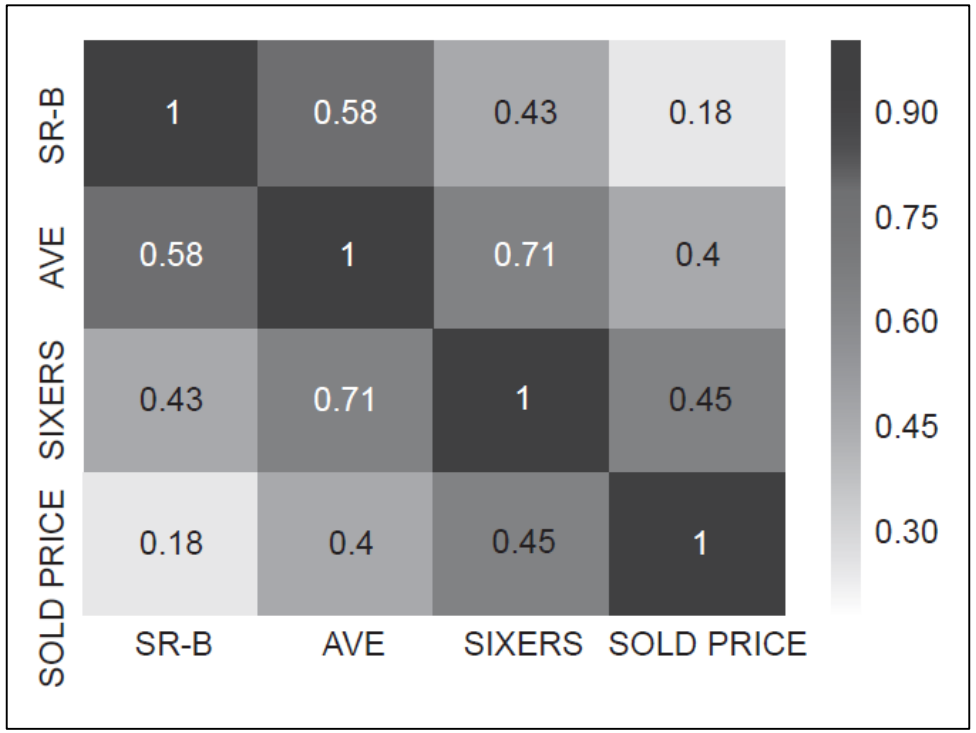


FIGURE 2.15 Heatmap of correlation values.

The color map scale is shown along the heatmap. Setting *annot* attribute to *true* prints the correlation values in each box of the heatmap and improves readability of the heatmap. Here the heatmap shows that *AVE* and *SIXER* show positive correlation, while *SOLD PRICE* and *SR-B* are not so strongly correlated.

[CHAPTER] 8

Design with Classes

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Programming languages that allow the programmer to define new classes of objects are called **object-oriented languages**. These languages also support a style of programming called **object-oriented programming**.

Getting Inside Objects and Classes

Like functions, objects are abstractions. A function packages an algorithm in a single operation that can be called by name. An object packages a set of data values—its state—and a set of operations—its methods—in a single entity that can be referenced with a name. This makes an object a more complex abstraction than a function. To get inside a function, you must view the code contained in its definition. To get inside an object, you must view the code contained in its class. A class definition is like a blueprint for each of the objects of that class. This blueprint contains

- Definitions of all of the methods that its objects recognize
- Descriptions of the data structures used to maintain the state of an object, or its attributes, from the implementer's point of view

EIU
EASTERN INTERNATIONAL UNIVERSITY

A First Example: The Student Class

A course-management application needs to represent information about students in a course. Each student has a name and a list of test scores. We can use these as the attributes of a class named **Student**. The **Student** class should allow the user to view a student's name, view a test score at a given position (counting from 1), reset a test score at a given position, view the highest test score, view the average test score, and obtain a string representation of the student's information. When a **Student** object is created, the user supplies the student's name and the number of test scores. Each score is initially presumed to be 0.

The interface or set of methods of the **Student** class is described in Table 9-1. Assuming that the **Student** class is defined in a file named **student.py**, the next session shows how it could be used:

```
>>> from student import Student
>>> s = Student("Maria", 5)
>>> print(s)
Name: Maria
Scores: 0 0 0 0 0
>>> s.setScore(1, 100)
>>> print(s)
Name: Maria
Scores: 100 0 0 0 0
>>> s.getHighScore()
100
>>> s.getAverage()
20
>>> s.getScore(1)
100
>>> s.getName()
'Maria'
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Table 9-1 The interface of the **Student** class

Student Method	What It Does
`s = Student(name, number)`	Returns a **Student** object with the given **name** and **number** of scores. Each score is initially 0.
`s.getName()`	Returns the student's name.
`s.getScore(i)`	Returns the student's *i*th score, **i** must range from 1 through the number of scores.
`s.setScore(i, score)`	Resets the student's *i*th score to score, **i** must range from 1 through the number of scores.
`s.getAverage()`	Returns the student's average score.
`s.getHighScore()`	Returns the student's highest score.
`s.__str()__`	Same as **str(s)**. Returns a string representation of the student's information.

EIU
EASTERN INTERNATIONAL UNIVERSITY

the syntax of a simple class definition is the following:

```
class <class name>(<parent class name>):
    <method definition-1>
    ...
    <method definition-n>
```

The class definition syntax has two parts: a class header and a set of method definitions that follow the class header. The class header consists of the class name and the parent class name.

The class name is a Python identifier. Although built-in type names are not capitalized, Python programmers typically capitalize their own class names to distinguish them from variable names.

The parent class name refers to another class. All Python classes, including the built-in ones, are organized in a tree-like **class hierarchy**. At the top, or root, of this tree is the most abstract class, named **`object`**, which is built in. Each class immediately below another class in the hierarchy is referred to as a **subclass**, whereas the class immediately above it, if there is one, is called its **parent class**. If the parenthesized parent class name is omitted from the class definition, the new class is automatically made a subclass of **`object`**.

The code for the **Student** class follows, and its structure is explained in the next few subsections:

```
"""
File: student.py
Resources to manage a student's name and test scores.
"""
class Student(object):
    """Represents a student."""

def  __init__(self, name, number):
    """Constructor creates a Student with the given
    name and number of scores and sets all scores
    to 0."""
    self.name = name
    self.scores = []
    for count in range(number):
        self.scores.append(0)

def getName(self):
    """Returns the student's name."""
    return self.name

def setScore(self, i, score):
    """Resets the ith score, counting from 1."""
    self.scores[i - 1] = score

def getScore(self, i):
    """Returns the ith score, counting from 1."""
    return self.scores[i - 1]

def getAverage(self):
    """Returns the average score."""
    return sum(self.scores) / len(self.scores)

def getHighScore(self):
    """Returns the highest score."""
    return max(self.scores)

def __str__(self):
    """Returns the string representation of the
    student."""
    return "Name: " + self.name + "\nScores: " + \
           " ".join(map(str, self.scores))
```

Method Definitions

each method definition must include a first parameter named **self**, even if that method seems to expect no arguments when called. When a method is called with an object, the interpreter binds the parameter **self** to that object so that the method's code can refer to the object by name.

Thus, for example, the code **s.getScore(4)** binds the parameter **self** in the method **getScore** to the **Student** object referenced by the variable **s**. The code for **getScore** can then use **self** to access that individual object's test scores.

The __init__ Method and Instance Variables

Most classes include a special method named **__init__**. Here is the code for this method in the **Student** class:

```
def __init__(self, name, number):
    """All scores are initially 0."""
    self.name = name
    self.scores = []
    for count in range(number):
        self.scores.append(0)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Note that **`__init__`** must begin and end with two consecutive underscores. This method is also called the class's **constructor**, because it is run automatically when a user instantiates the class. Thus, when the code segment

```
s = Student("Juan", 5)
```

is run, Python automatically runs the constructor or **`__init__`** method of the **`Student`** class. The purpose of the constructor is to initialize an individual object's attributes. In addition to **`self`**, the **`Student`** constructor expects two arguments that provide the initial values for these attributes. From this point on, when we refer to a class's constructor, we mean its **`__init__`** method.

The attributes of an object are represented as **instance variables**. Each individual object has its own set of instance variables. These variables serve as storage for its state. The scope of an instance variable (including **`self`**) is the entire class definition. Thus, all of the class's methods are in a position to reference the instance variables. The lifetime of an instance variable is the lifetime of the enclosing object.

Within the class definition, the names of instance variables must begin with **`self`**. For example, in the definition of the **`Student`** class, the instance variables **`self.name`** and **`self.scores`** are initialized to a string and a list, respectively.

The `__str__` Method

Many built-in Python classes usually include an **`__str__`** method. This method builds and returns a string representation of an object's state. When the **`str`** function is called with an object, that object's **`__str__`** method is automatically invoked to obtain the string that **`str`** returns. For example, the function call **`str(s)`** is equivalent to the method call **`s.__str__()`**, and is simpler to write. The function call **`print(s)`** also automatically runs **`str(s)`** to obtain the object's string representation for output.

Here is the code for the **`__str__`** method in the **`Student`** class:

```
def __str__(self) :
    """Returns the string representation of the student."""
    return "Name: " + self.name + "\nScores: " + \
           " ".join(map(str, self.scores))
```

Accessors and Mutators

Methods that allow a user to observe but not change the state of an object are called **accessors**. Methods that allow a user to modify an object's state are called **mutators**. The **`Student`** class has just one mutator method. It allows the user to reset a test score at a given position. The remaining methods are accessors.

Here is the code for the mutator method **`setScore`**:

```
def setScore(self, i, score):
    """Resets the ith score, counting from 1."""
    self.scores[i - 1] = score
```

The Lifetime of Objects

In Python, an object becomes a candidate for the graveyard when the program that created it can no longer refer to it.

For example, the next session creates two references to the same **Student** object:

The strange-looking code in angle brackets is what Python displays when it prints this type of object in the shell. As long as one of these references survives, the **Student** object can remain alive.

```
>>> s = Student("Sam", 10)
>>> cscilll = [s]
>>> cscilll
[<__main__.Student instance at 0x11ba2b0>]
>>> s
<__main__.Student instance at 0x11ba2b0>
```

Continuing this session, we now sever both references to the **Student** object:

The **Student** object still exists, but the Python virtual machine will eventually recycle its storage during a process called **garbage collection**. For all intents and purposes, this object has expired, and its storage will eventually be used to create other objects.

```
>>> s = None
>>> cscilll.pop()
<__main__.Student instance at 0x11ba2b0>
>>> print(s)
None
>>> cscilll
[]
```

Rules of Thumb for Defining a Simple Class

We conclude this section by listing several rules of thumb for designing and implementing a simple class:

1. Before writing a line of code, think about the behavior and attributes of the objects of the new class. What actions does an object perform, and how, from the external perspective of a user, do these actions access or modify the object's state?

2. Choose an appropriate class name, and develop a short list of the methods available to users. This interface should include appropriate method names and parameter names, as well as brief descriptions of what the methods do. Avoid describing how the methods perform their tasks.

3. Write a short script that appears to use the new class in an appropriate way. The script should instantiate the class and run all of its methods. Of course, you will not be able to execute this script until you have completed the next few steps, but it will help to clarify the interface of your class and serve as an initial test bed for it.

Rules of Thumb for Defining a Simple Class

4. Choose the appropriate data structures to represent the attributes of the class. These will be either built-in types such as integers, strings, and lists, or other programmer-defined classes.

5. Fill in the class template with a constructor (an **`__init__`** method) and an **`__str__`** method. Remember that the constructor initializes an object's instance variables, whereas **`__str__`** builds a string from this information. As soon as you have defined these two methods, you can test your class by instantiating it and printing the resulting object.

6. Complete and test the remaining methods incrementally, working in a bottom-up manner. If one method depends on another, complete the second method first.

7. Remember to document your code. Include a docstring for the module, the class, and each method. Do not add docstrings as an afterthought. Write them as soon as you write a class header or a method header. Be sure to examine the results by running **`help`** with the class name.

Structuring Classes with Inheritance and Polymorphism

Object-based programming involves the use of objects, classes, and methods to solve problems. Object-oriented programming requires the programmer to master the following additional concepts:

1. **Data encapsulation**. Restricting the manipulation of an object's state by external users to a set of method calls.
2. **Inheritance**. Allowing a class to automatically reuse and extend the code of similar but more general classes.
3. **Polymorphism**. Allowing several different classes to use the same general method names.

Although Python is considered an object-oriented language, its syntax does not enforce data encapsulation. As you have seen, in the case of simple container objects, like playing cards, with little special behavior, it is handy to be able to access the objects' data without a method call.

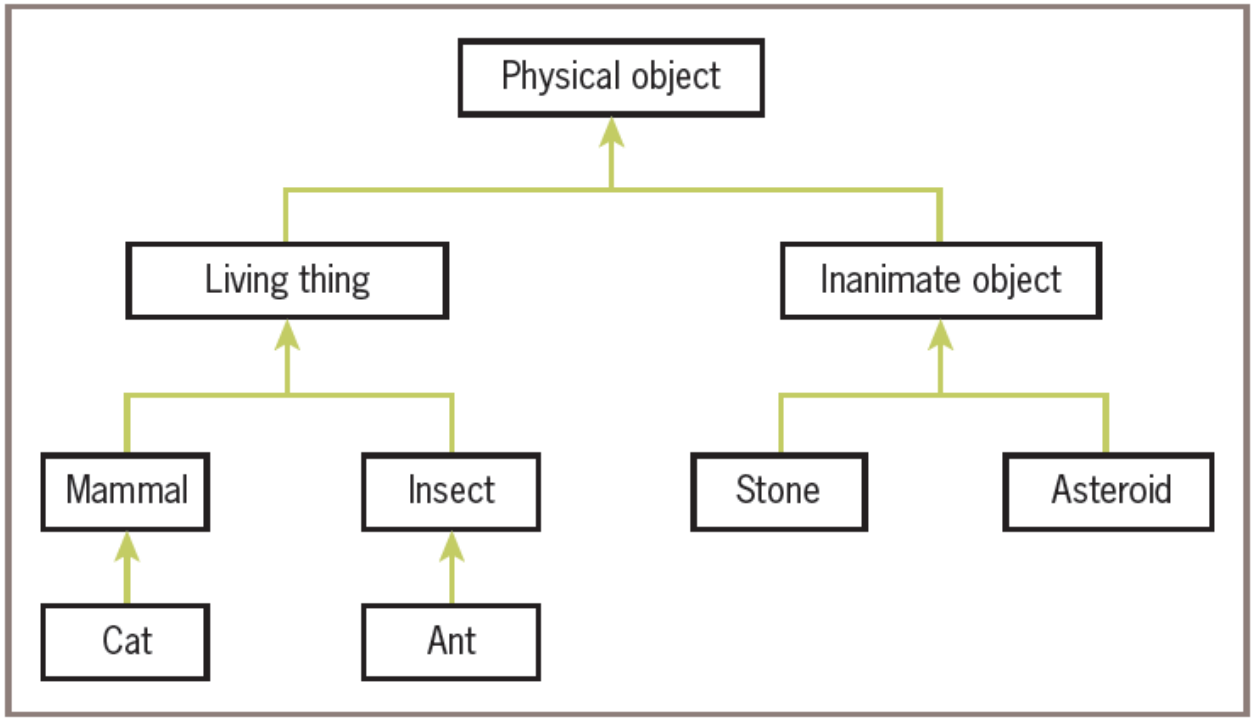
Unlike data encapsulation, inheritance and polymorphism are built into Python's syntax.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Inheritance Hierarchies and Modeling

Objects in the natural world and objects in the world of artifacts can be classified using **inheritance hierarchies**. A simplified hierarchy of natural objects is depicted in Figure 9-5.

Figure 9-5 A simplified hierarchy of objects in the natural world



EIU
EASTERN INTERNATIONAL UNIVERSITY

At the top of a hierarchy is the most general class of objects. This class defines features that are common to every object in the hierarchy. For example, every physical object has a mass. Classes just below this one have these features as well as additional ones. Thus, a living thing has a mass and can also grow and die. The path from a given class back up to the topmost one goes through all of that given class's ancestors. Each class below the topmost one inherits attributes and behaviors from its ancestors and extends these with additional attributes and behavior.

An object-oriented software system models this pattern of inheritance and extension in real-world systems by defining classes that extend other classes. In Python, all classes automatically extend the built-in **`object`** class, which is the most general class possible.

it is possible to extend any existing class using the syntax:

```
class <new class name>(<existing parent class name>):
```

Thus, for example, **`PhysicalObject`** would extend **`object`**, **`LivingThing`** would extend **`PhysicalObject`**, and so on.

The real advantage of inheritance in a software system is that each new subclass acquires all of the instance variables and methods of its ancestor classes for free. Like function definitions and class definitions, inheritance hierarchies provide an abstraction mechanism that allows the programmer to avoid reinventing the wheel or writing redundant code.

Polymorphic Methods

a subclass inherits data and methods from its parent class. We would not bother subclassing unless the two classes shared a substantial amount of **abstract behavior**. By this term, we mean that the classes have similar sets of methods or operations. A subclass usually adds something extra, such as a new method or a data attribute, to the ensemble provided by its superclass.

In some cases, the two classes have the same interface, or set of methods available to external users. In these cases, one or more methods in a subclass override the definitions of the same methods in the superclass to provide specialized versions of the abstract behavior. Like any object-oriented language, Python supports this capability with **polymorphic methods**. The term *polymorphic* means "many bodies," and it applies to two methods that

The `__str__` method is a good example of a polymorphic method that appears throughout Python's system of classes.

EIU
EASTERN INTERNATIONAL UNIVERSITY





Advanced Python Programming

EIU
EASTERN INTERNATIONAL UNIVERSITY